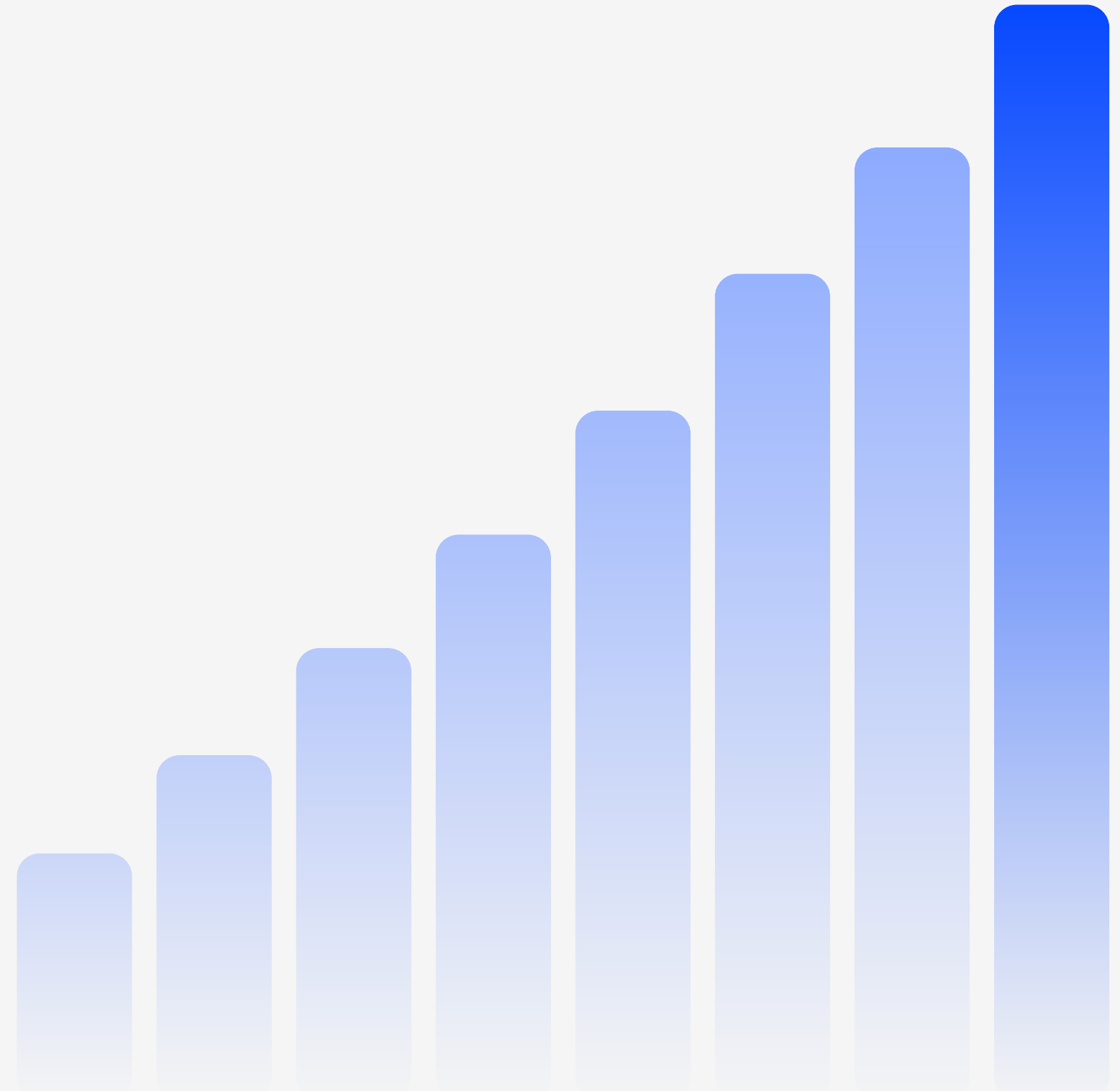


공공데이터 융합 풀스택 개발자 양성과정

애플리케이션 설계



목 차

01

DataSourceConfig 강현욱

02

GlobalExceptionHandler 정승원

03

MemberController 박형규

04

MemberDAO 김정희

05

MemberDTO 강현욱

06

구현화면

01 | 코드구현 - DataSourceConfig 강현욱

@Configuration ㄹ kanghyunuk *

public class DataSourceConfig {

```
// TODO 1) @Bean dataSource2() : BasicDataSource(commons-dbc2)
//      - setDriverClassName("com.mysql.cj.jdbc.Driver")
//      - setUrl("jdbc:mysql://localhost:3306/testdb"), setUsername("root"), s
```

@Bean 신규 *

public DataSource dataSource2() {

```
    BasicDataSource dataSource = new BasicDataSource();
    dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
    dataSource.setUrl("jdbc:mysql://localhost:3306/testdb");
    dataSource.setUsername("root");
    dataSource.setPassword("1234");
    return dataSource;
}
```

```
// TODO 2) @Bean dataSource3() : HikariDataSource
```

```
//      - setDriverClassName("com.mysql.cj.jdbc.Driver")
//      - setJdbcUrl("jdbc:mysql://localhost:3306/testdb"), setUsername("root")
//      * MemberDAO 는 dataSource3(HikariDataSource) 를 주입받아 사용한다.
```

@Bean ㄹ kanghyunuk

public HikariDataSource dataSource3() {

```
    HikariDataSource dataSource = new HikariDataSource();
    dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
    dataSource.setJdbcUrl("jdbc:mysql://localhost:3306/testdb");
    dataSource.setUsername("root");
    dataSource.setPassword("1234");
    return dataSource;
}
```

dataSource2 : Apache Commons DBCP2

- 드라이버 클래스 지정 (com.mysql.cj.jdbc.Driver)
- 연결 URL 지정 (DBCP2 규격 메서드인 setUrl() 사용)
- 계정 정보 입력 (root / 1234)

dataSource3 : HikariCP

- 드라이버 클래스 지정 (com.mysql.cj.jdbc.Driver)
- 연결 URL 지정 (HikariCP 전용 메서드인 setJdbcUrl() 사용)
- 계정 정보 입력 (root / 1234)

02 | 코드구현 - GlobalExceptionHandler-전역예외처리

```
@ControllerAdvice @jungwoogyun *
public class GlobalExceptionHandler {

    // TODO 1) @ExceptionHandler(FileNotFoundException.class) -> model 에 담고 "global/error1" 반환
    @ExceptionHandler(FileNotFoundException.class) 신규 *
    public String exceptionHandler_1 (Exception e, Model model) {
        model.addAttribute( attributeName: "e", e);
        return "global/error1";
    }

    // TODO 2) @ExceptionHandler(ArithmeticException.class) -> "global/error2" 반환
    @ExceptionHandler(ArithmeticException.class) 신규 *
    public String exceptionHandler_2 (Exception e, Model model) {
        model.addAttribute( attributeName: "e", e);
        return "global/error2";
    }

    // TODO 3) @ExceptionHandler(Exception.class) -> "global/error3" 반환
    @ExceptionHandler(Exception.class) 신규 *
    public String exceptionHandler_3 (Exception e, Model model) {
        model.addAttribute( attributeName: "e", e);
        return "global/error3";
    }
}
```

이 클래스는 @ControllerAdvice를 사용한 전역 예외 처리 클래스입니다.

일반적으로 Controller에서 예외가 발생하면 각 Controller마다 try-catch를 작성해야 하지만, @ControllerAdvice를 사용하면 프로젝트 전체에서 발생하는 예외를 한 곳에서 관리할 수 있습니다.

Controller 실행 중 예외가 발생하면 Spring이 해당 예외를 처리할 수 있는 @ExceptionHandler 메서드를 찾아 자동으로 실행하고, 지정한 에러 페이지를 사용자에게 보여줍니다.

```
// TODO 1) @ExceptionHandler(FileNotFoundException.class) -> model 에 담고 "global/error1" 반환
@ExceptionHandler(FileNotFoundException.class) 신규 *
public String exceptionHandler_1 (Exception e, Model model) {
    model.addAttribute(attributeName: "e", e);
    return "global/error1";
}
```

첫 번째로 FileNotFoundException을 처리하는 예외 핸들러입니다.

@ExceptionHandler(FileNotFoundException.class)는 파일을 찾을 수 없는 예외가 발생했을 때 이 메서드를 실행하라는 의미입니다.

메서드의 Exception e는 발생한 예외 객체를 전달받기 위한 매개변수이고, Model model은 예외 정보를 화면으로 전달하기 위한 객체입니다.

model.addAttribute("e", e)를 통해 예외 객체를 View에 전달하고, 마지막에 "global/error1"을 반환하여 error1 페이지로 이동시킵니다.

즉, 파일 관련 예외가 발생하면 예외 정보를 담아서 error1 화면을 보여주는 역할을 합니다.


```
// TODO 2) @ExceptionHandler(ArithmeticException.class) -> "global/error2" 반환
@ExceptionHandler(ArithmeticException.class) 신규 *
public String exceptionHandler_2 (Exception e, Model model) {
    model.addAttribute(attributeName: "e", e);
    return "global/error2";
}
```

두 번째로는 `ArithmeticException`을 처리하는 예외 핸들러입니다.

`ArithmeticException`은 0으로 나누기 같은 잘못된 산술 연산이 발생했을 때 발생하는 예외입니다.

예외가 발생하면 Spring이 이 메서드를 자동으로 호출하고, 예외 객체를 Model에 저장한 후 "global/error2"를 반환하여 error2 페이지로 이동시킵니다.

즉, 계산 과정에서 발생한 예외를 처리하여 사용자에게 별도의 에러 화면을 제공하는 역할을 합니다.

```
// TODO 3) @ExceptionHandler(Exception.class)      -> "global/error3" 반환
@ExceptionHandler(Exception.class) 신규 *
public String exceptionHandler_3 (Exception e, Model model) {
    model.addAttribute(attributeName: "e", e);
    return "global/error3";
}
```

세 번째로는 모든 예외를 처리하는 최종 예외 핸들러입니다.

@ExceptionHandler(Exception.class)는 Exception을 상속받는 모든 예외를 처리할 수 있습니다.

앞에서 정의한 FileNotFoundException이나 ArithmeticException처럼 특정 예외 처리기가 존재하면 그 메서드가 먼저 실행되고, 해당하는 처리기가 없을 경우 이 메서드가 실행됩니다.

따라서 예상하지 못한 예외가 발생하더라도 프로그램이 바로 종료되지 않고 error3 페이지를 보여줄 수 있습니다.

즉, 프로젝트 전체의 예외를 안전하게 처리하기 위한 마지막 안전장치 역할을 수행합니다.

03 | 코드구현 - MemberController 박형규

```
// TODO: DAO 등에서 발생한 예외를 처리할 @ExceptionHandler 메서드를 작성하라.
// - (Exception e, Model model) 를 받아 e 메시지를 model 에 담고 "member/error" 뷰 반환
// - import : org.springframework.web.bind.annotation.ExceptionHandler
@ExceptionHandler 0개의 사용위치 parkhyeonggyu15
public String SQLExceptionHandler(Exception e, Model model){
    model.addAttribute( attributeName: "e", e.getMessage()); //에러 메시지를 받기 위해 Model에 저장
    return "member/error"; // "member/error"뷰 반환
}
```

1.컨트롤러에서 예외가 발생하면 @ExceptionHandler가 감지 후 처리.

2.그다음 매개변수로 들어온 Exception e를 통해 구체적인 에러 메시지를 꺼내고, 이 메시지를 사용자 화면에 보여주기 위해 Model 객체에 저장.

3.기본 에러 화면 대신 미리 만들어둔 member/error라는 커스텀 에러 페이지를 사용자에게 반환.

03| 코드구현 - MemberController 박형규

```
@PostMapping("/add") @ parkhyeonggyu15 +1
public String memberAddPost(@Valid MemberDTO memberDTO, BindingResult bindingResult,
                             Model model, RedirectAttributes redirectAttributes) throws SQLException {
    log.info("POST /member/add..." + memberDTO);

    // TODO:
    // 1) bindingResult.hasErrors() 이면 FieldError 들을 들며
    //    model.addAttribute(error.getField(), error.getDefaultMessage()) 후 "member/add" 반환
    //    (import org.springframework.validation.FieldError)
    // 2) 검증 통과 시 memberDAO.insert(memberDTO) 호출
    // 3) redirectAttributes.addFlashAttribute("message", "회원등록 성공!") 후 "redirect:/member/list" 반환
    if(bindingResult.hasErrors()) {
        // 1. 데이터 검증 오류 발생 시 처리
        // 발생한 모든 필드 에러를 순회하며 Model에 담아 화면으로 전달 (e.g., id -> "아이디를 입력하세요")
        for (FieldError error : bindingResult.getFieldErrors()) {
            model.addAttribute(error.getField(), error.getDefaultMessage());
        }
        return "member/add"; // 입력 폼 페이지로 돌아가기 (작성 중이던 데이터 및 에러 메시지 유지)
    }
    // 2. 비즈니스 로직 수행 (DB 삽입)
    int result = memberDAO.insert(memberDTO);
    // 3. 성공 알림 메시지 전달 및 리다이렉트
    // addFlashAttribute: 리다이렉트 직후 약 한 번만 세션을 통해 데이터를 전달함 (새로고침 시 소멸)
    redirectAttributes.addFlashAttribute(attributeName: "message", attributeValue: "회원등록 성공!");

    return "redirect:/member/list"; // 회원 목록 페이지로 리다이렉트
}
```

1. 사용자가 회원 등록을 요청하면 @Valid를 통해 입력된 데이터가 올바른지 검증부터 수행.

2. 이때 검증 과정에서 오류가 발견되어 bindingResult.hasErrors()가 참이 되면, 오류가 발생한 모든 필드를 하나씩 돌면서 에러 메시지를 Model에 저장.

3. 작업 중이던 데이터 및 에러 메시지를 유지하면서 입력 폼 페이지로 반환

4. 만약 모든 검증을 무사히 통과하면 실제 비즈니스 로직이 실행되면서 memberDAO.insert()를 통해 데이터베이스에 회원 정보 저장.

5. 등록이 성공적으로 완료되면 redirectAttributes.addFlashAttribute를 사용해 "회원등록 성공!"이라는 알림 메시지를 세션에 임시로 저장 후 회원 목록 페이지로 리다이렉트.

04 | 코드구현 - memberDAO 김정희

```
@Repository
public class MemberDAO {

    @Autowired
    private DataSource dataSource3; // DataSourceConfig 의 HikariDataSource 빈

    // TODO: 회원 1건 등록
    // - dataSource3.getConnection() 으로 Connection 획득
    // - "insert into tbl_member values(null,?,?,?,?)" PreparedStatement 생성
    // - name / email / phone / createdAt(Timestamp.valueOf(LocalDateTime.now())) 바인딩
    // - executeUpdate() 결과(int) 반환
    public int insert(MemberDTO dto) throws SQLException {
        Connection conn = dataSource3.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql: "INSERT INTO tbl_member VALUES (null,?,?,?,?)");
        pstmt.setString(parameterIndex: 1, dto.getName());
        pstmt.setString(parameterIndex: 2, dto.getEmail());
        pstmt.setString(parameterIndex: 3, dto.getPhone());
        pstmt.setTimestamp(parameterIndex: 4, Timestamp.valueOf(LocalDateTime.now()));
        int result = pstmt.executeUpdate();
        return result;
    }
}
```

@Repository를 통해 MemberDAO가 repository임을 명시

@Autowired를 통해 dataSource3(HikariDataSource) 를 주입 받아 연결

회원 등록 함수 insert로 MemberDTO 단위로 매개변수를 받고 dataSource3.getConnection() 으로 Connection 획득

PreparedStatement를 사용하며 SQL문으로 tbl_member테이블에 5개의 값을 순서대로 받아서 저장하도록 작성하는데 이때 첫번째 값인 id는 DB상으로 auto increment를 사용하여 값을 얻기 때문에 null로 생략

이후 바인딩 작업을 하는데 나머지는 String타입으로 받지만 createdAt은 DTO에서는 LocalDateTime 형태로 받았지만 이를 DB에 저장하기 위해 Timestamp형태로 변환

마지막으로 executeUpdate()로 DB에 insert하며 이때 저장된 개수를 int로 받고 반환

04 | 코드구현 - memberDAO 김정희

```
// TODO: 회원 전체 조회
// - "select * from tbl_member order by id desc" 실행
// - ResultSet 을 들어 MemberDTO.builder() 로 매핑하여 List 로 반환
public List<MemberDTO> selectAll() throws SQLException {
    Connection conn = dataSource3.getConnection();
    PreparedStatement pstmt = conn.prepareStatement("SELECT * FROM tbl_member ORDER BY id DESC");
    ResultSet rs = pstmt.executeQuery();
    List<MemberDTO> list = new ArrayList<>();
    MemberDTO dto = null;
    while (rs.next()){
        dto = MemberDTO.builder()
            .id(rs.getLong( columnLabel: "id"))
            .name(rs.getString( columnLabel: "name"))
            .email(rs.getString( columnLabel: "email"))
            .phone(rs.getString( columnLabel: "phone"))
            .createAt(rs.getTimestamp( columnLabel: "createAt").toLocalDateTime())
            .build();
        list.add(dto);
    }
    return list;
}
```

회원 전체 조회 함수 `selectAll`로 아무런 조건없이 모든 회원을 불러오기 때문에 매개변수를 받지 않으며 `dataSource3.getConnection()`으로 Connection 획득

`PreparedStatement`를 사용하며 SQL문으로 `tbl_member`테이블의 모든 데이터를 내림차순으로 정렬하여 조회하며 `executeQuery()`를 사용하여 데이터를 가지고 와 `ResultSet`타입에 저장

`ResultSet`의 데이터를 `MemberDTO`단위로 저장하기 위해 `List<MemberDTO>` 형태로 여러개의 `MemberDTO`를 하나의 `list`에 담으며 이를 보조하기 위해 임시로 `MemberDTO`단위로 저장할 수 있게 오브젝트 변수를 `null`로 초기화

`while(rs.next())`로 `rs`의 다음이 없을 때 까지 반복하여 임시 오브젝트 변수에 `builder`를 사용하여 저장 후 리스트에 추가하는데 이 때 `DTO`는 `createAt`이 `LocalDateTime` 타입이기에 변환해서 저장

반복문이 끝나면 `list` 반환

05 | 코드구현 - MemberDTO 강현욱

```
// TODO: 아래 클래스에 Lombok 애너테이션을 붙여 getter/setter/builder
@Slf4j 8개 사용 위치 ㄹ kanghyunuk
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class MemberDTO {

    // TODO: 각 필드에 알맞은 검증 애너테이션을 추가하라.
    private Long id;                // 식별자 (검증 없음, DB AUTO_)
    @NotBlank
    private String name;            // TODO: @NotBlank
    @NotBlank
    @Email
    private String email;           // TODO: @NotBlank + @Email
    @NotBlank
    private String phone;           // TODO: @NotBlank
    private LocalDateTime createdAt; // 등록일시 (검증 없음)
}
```

@Slf4j

- 로그 출력을 위한 log 객체를 자동으로 생성

@Data

- @Getter, @Setter, @ToString, @EqualsAndHashCode, @RequiredArgsConstructor를 한 번에 생성하여 DTO의 기본 기능을 완성

@NoArgsConstructor

- 기본 생성자를 자동으로 생성

@AllArgsConstructor

- 전체 필드를 매개변수로 받는 생성자를 생성 (Builder 패턴 사용을 위한 필수 조건)

@Builder

- 빌더 패턴을 적용하여 안전한 객체 생성을 지원

member/list

-- 회원목록 출력

```
1 • SELECT * FROM testdb.tbl_member;
```

Result Grid					
Filter Rows:					
Edit:   					
Export/Import					
	id	name	email	phone	createAt
	1	김철수	kim@example.com	010-1111-2222	2026-06-05 15:33:21
	2	이영희	lee@example.com	010-3333-4444	2026-06-05 15:33:21
	3	박민수	park@example.com	010-5555-6666	2026-06-05 15:33:21
	NULL	NULL	NULL	NULL	NULL

PAGE : /member/list

[회원등록](#)

ID	NAME	EMAIL	PHONE	CREATEAT
3	박민수	park@example.com	010-5555-6666	2026-06-05T15:33:21
2	이영희	lee@example.com	010-3333-4444	2026-06-05T15:33:21
1	김철수	kim@example.com	010-1111-2222	2026-06-05T15:33:21

PAGE : /member/add

이름 공백일 수 없습니다
이메일 공백일 수 없습니다
연락처 공백일 수 없습니다

PAGE : /member/add

이름
이메일 올바른 형식의 이메일 주소여야 합니다
연락처

[회원목록](#)

member/add

-- 회원목록 추가

- **공백 검증 확인**
- **@EMAIL 검증 확인**

PAGE : /member/add

이름 공백일 수 없습니다
이메일 공백일 수 없습니다
연락처 공백일 수 없습니다

[회원목록](#)

PAGE : /member/list

[회원등록](#)

ID	NAME	EMAIL	PHONE	CREATEAT
7	홍길동	aaa@naver.com	010-0000-0000	2026-06-05T16:15:18
3	박민수	park@example.com	010-5555-6666	2026-06-05T15:33:21
2	이영희	lee@example.com	010-3333-4444	2026-06-05T15:33:21
1	김철수	kim@example.com	010-1111-2222	2026-06-05T15:33:21

회원등록 성공!

member/add

-- 회원목록 추가

- 정보입력
- 회원저장 완료 및 저장된 회원 리스트 출력